

Reviewer Pack — Minimal Order of Semigroup Generators and Circuit Monotone W...

Entry 19e716713324 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 20:13:18 UTC

Minimal Order of Semigroup Generators and Circuit Monotone Width Inequality

Entry ID: 19e716713324 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 20:13:18 UTC

1. Conjecture

Field A (mathematical branch): Semigroup Theory **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For any given boolean circuit C with n inputs, the minimal order of the monoid generators of the semigroup generated by C is at most a constant times the circuit monotone width $w(C)$ of C , i.e., $|S(C)| \leq \kappa \cdot w(C)$, where κ is a fixed constant.

Rationale (proposer's reasoning):

Semigroup theory provides a framework for studying algebraic structures that can be used to encode properties of boolean functions. The minimal order of semigroup generators has been studied in combinatorial group theory but not extensively in the context of circuit complexity. By connecting this invariant to circuit monotone width, we may uncover new insights into the structure of boolean circuits and their computational hardness.

Taxonomy category: semigroup_algebraic_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 498a8514f7ba4934

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all circuits C , the ratio of minimal order of monoid generators $|S(C)|$ to monotone width $w(C)$ is less than or equal to a constant κ ($\kappa \leq 1$), and falsified if any circuit C has a ratio greater than κ .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        # Generate a random boolean circuit with n inputs
        if n == 1:
            return [[0], [1]]
        else:
            subcircuits = [generate_circuit(n // 2) for _ in range(2)]
            return [subcircuit + [n] for subcircuit in subcircuits]

    def compute_semgroup_size(circuit):
        # Compute the minimal order of the monoid generators |S(C)|
        n = len(circuit)
        if n == 1:
            return 1
        else:
            semgroup = set()
            for i in range(2 ** (n - 1)):
                state = [0] * (n - 1)
                for j in range(n):
                    if circuit[j][-1] == 1:
                        state[(i >> j) & 1] ^= 1
                semgroup.add(tuple(state))
            return len(semgroup)

    def compute_circuitmonowidth(circuit):
        # Compute the monotone width w(C)
        n = len(circuit)
        if n == 1:
            return 1
        else:
            width = 0
```

```

        for i in range(2 ** (n - 1)):
            state = [0] * (n - 1)
            for j in range(n):
                if circuit[j][-1] == 1:
                    state[(i >> j) & 1] ^= 1
            width = max(width, sum(state))
        return width

n_values = [5, 10, 15, 20, 30, 40]
total_semgroup_size = 0
total_circuitmonowidth = 0
instances_tested = 0

for n in n_values:
    for _ in range(5): # Test each size 5 times
        circuit = generate_circuit(n)
        semgroup_size = compute_semgroup_size(circuit)
        circuitmonowidth = compute_circuitmonowidth(circuit)
        if circuitmonowidth > 0: # Avoid division by zero
            total_semgroup_size += semgroup_size
            total_circuitmonowidth += circuitmonowidth
            instances_tested += 1

if total_circuitmonowidth == 0:
    return {
        "metric_name": "Semigroup Size / Circuit Monotone Width",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "Circuit Monotone Width is zero"
    }

ratio = Fraction(total_semgroup_size, total_circuitmonowidth)
return {
    "metric_name": "Semigroup Size / Circuit Monotone Width",
    "metric_value": float(ratio),
    "instances_tested": instances_tested,
    "n_max": max(n_values),
    "conjecture_holds": ratio <= 1,
    "counterexample": "" if ratio <= 1 else f"Ratio: {ratio}, Expected: ≤ 1"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0

```

```

    print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"Ratio exceeded 1\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

False, 'counterexample': 'Circuit Monotone Width is zero'}
TRIAL: {'metric_name': 'Semigroup Size / Circuit Monotone Width', 'metric_value': None, 'instances_tested': 5}
TRIAL: {'metric_name': 'Semigroup Size / Circuit Monotone Width', 'metric_value': None, 'instances_tested': 5}
TRIAL: {'metric_name': 'Semigroup Size / Circuit Monotone Width', 'metric_value': None, 'instances_tested': 5}
TRIAL: {'metric_name': 'Semigroup Size / Circuit Monotone Width', 'metric_value': None, 'instances_tested': 5}
TRIAL: {'metric_name': 'Semigroup Size / Circuit Monotone Width', 'metric_value': None, 'instances_tested': 5}
TRIAL: {'metric_name': 'Semigroup Size / Circuit Monotone Width', 'metric_value': None, 'instances_tested': 5}
TRIAL: {'metric_name': 'Semigroup Size / Circuit Monotone Width', 'metric_value': None, 'instances_tested': 5}
TRIAL: {'metric_name': 'Semigroup Size / Circuit Monotone Width', 'metric_value': None, 'instances_tested': 5}
TRIAL: {'metric_name': 'Semigroup Size / Circuit Monotone Width', 'metric_value': None, 'instances_tested': 5}
TRIAL: {'metric_name': 'Semigroup Size / Circuit Monotone Width', 'metric_value': None, 'instances_tested': 5}
TRIAL: {'metric_name': 'Semigroup Size / Circuit Monotone Width', 'metric_value': None, 'instances_tested': 5}

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code only tests up to $n = 40$ and uses a very small number of instances per size (5). This is insufficient to confirm the conjecture as it may not scale with larger circuits. Additionally, the metric ‘circuit monotone width’ could be trivially zero for certain circuits, which would make the ratio unbounded by construction.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that for at least one seed (first_failing_seed=11), the ratio of minimal order of monoid generators to monotone width exceed | next: Further investigation is needed to determine if the counterexample holds for larger circuits and under different conditions. Additionally, exploring alternative metrics or methods to calculate the monotone width could provide more insights into the validity of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14701
2	preregistration	ollama_remote	glm4:latest	0	0	9615
3	novelty	ollama_remote	glm4:latest	0	0	9166

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	8783
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19018
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9382
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11350
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12731
9	critic	ollama_remote	glm4:latest	0	0	14308
10	judge	ollama_remote	glm4:latest	0	0	9397

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 118452 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/19e716713324.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/19e716713324.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/19e716713324.tar.gz* (if generated)